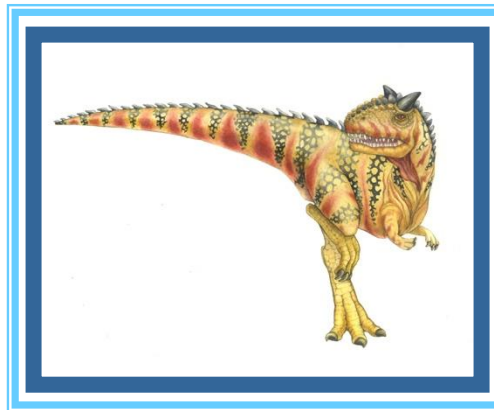


Threads





¿Qué es un Thread?

- ¿Qué es un Thread, cual puede ser su relación con un proceso?
- ¿En que otras terminologías relacionadas con la computación encontramos el termino Thread?





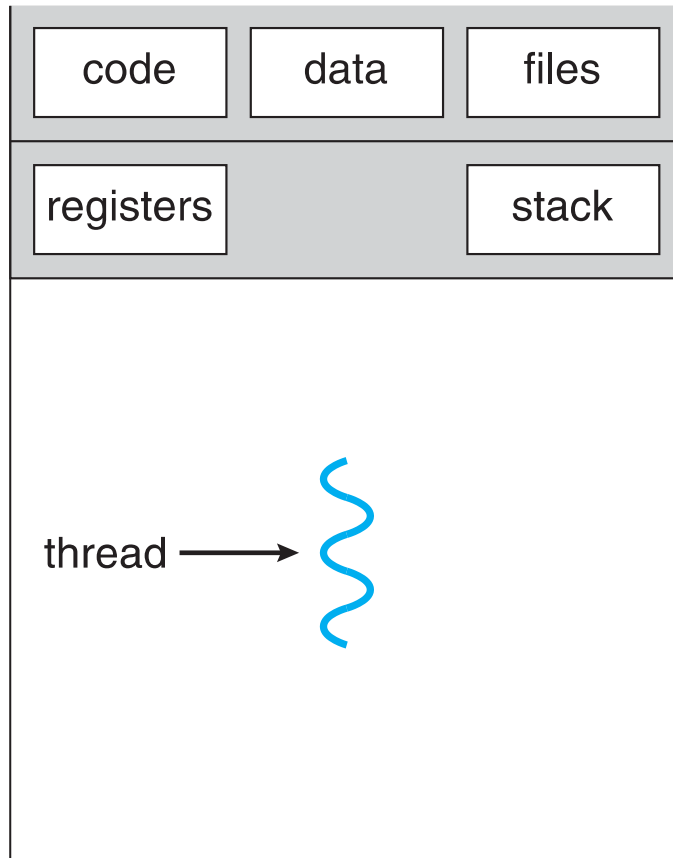
¿Qué es un *thread*?

- Un proceso cuenta con registros que contienen información en uso, una pila de ejecución y un *program counter*.
 - Esto conforma su hilo de ejecución o *thread*.
 - El sistema operativo maneja al menos un *thread* por proceso, y son los *threads* los que se calendarizan para procesamiento.
- Creación de procesos para concurrencia es más tardado que creación de *threads*.
 - Los *threads* comparten espacio de memoria, código fuente, datos globales y archivos del proceso.
- Cada *thread* suele poseer información característica, como un estado (*ready*, *blocked*, etc.) y un identificador.

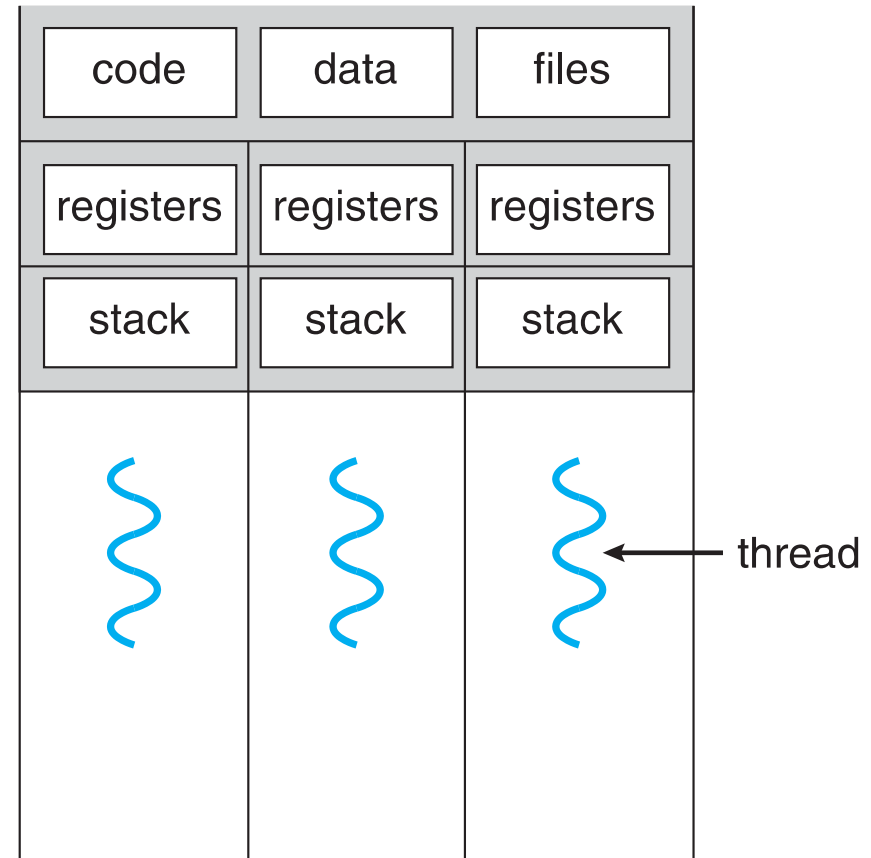




Procesos con uno y múltiples *threads*



single-threaded process



multithreaded process





Beneficios

- ❑ **Responsividad** – Permiten continuar la ejecución ante llamadas a sistema bloqueadoras (trabajo en *foreground* y *background*).
- ❑ **Recursos compartidos** – *threads* no tienen necesidad de paso de mensajes porque comparten memoria.
- ❑ **Más económicos** – toma menos tiempo crear y matar *threads* que procesos, y sus cambios de contexto son menos pesados.
- ❑ **Escalabilidad** – aprovechan el potencial del paralelismo, lo que permite incrementar las operaciones por unidad de tiempo.
- ❑ **Modularidad en diseño** – puede ser más fácil diseñar e implementar programas al usar *threads* para separar sus actividades.





Procesos de Linux para manejo de *threads*

- ❑ ¿Qué diferencias hay entre los conceptos de *thread* de *kernel* y *thread* de usuario?
- ❑ ¿Qué hace el proceso de *kernel* `kthreadd`, en Linux? ¿Por qué tiene el identificador 2?
- ❑ Al ejecutar el comando `top` en una terminal de Linux veremos varios procesos llamados `kworker`. ¿Qué son estos procesos y qué significan los números que aparecen a la par?
- ❑ Describa otros procesos de Linux destinados a manejar *threads* de *kernel*, como `khelper` y `ksoftirqd`.

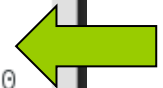




kthreadd

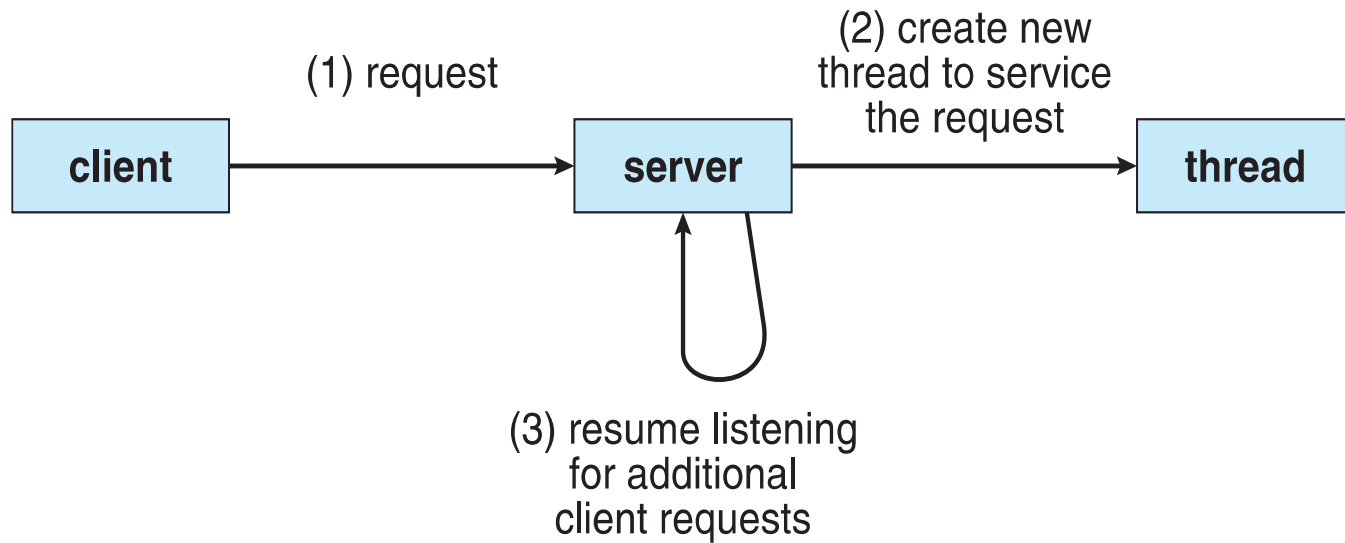
```
oscreader@OSC: ~/Desktop/Labs/2
File Edit View Search Terminal Help
top - 06:27:04 up 3:09, 2 users, load average: 0.00, 0.00, 0.00
Tasks: 134 total, 1 running, 133 sleeping, 0 stopped, 0 zombie
%Cpu(s): 1.0 us, 0.7 sy, 0.0 ni, 98.3 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
KiB Mem: 771744 total, 683952 used, 87792 free, 33656 buffers
KiB Swap: 391164 total, 116836 used, 274328 free. 239444 cached Mem

  PID USER      PR  NI   VIRT   RES   SHR  S  %CPU  %MEM    TIME+  COMMAND
 1064 oscread+  20   0 434888 129060 48276 S   0.7  16.7   9:18.32 gnome-shell
   522 root      20   0  11216  29204  8452 S   0.3   3.8   2:26.09 Xorg
 2456 oscread+  20   0   72364  26688 20688 S   0.3   3.5   0:08.84 gnome-termi+
 4034 oscread+  20   0   5080   2732  2368 R   0.3   0.4   0:00.03 top
     1 root      20   0   23948   3196  2176 S   0.0   0.4   0:01.15 systemd
     2 root      20   0        0        0        0 S   0.0   0.0   0:00.00 kthreadd
     3 root      20   0        0        0        0 S   0.0   0.0   0:00.37 ksoftirqd/0
     5 root        0 -20        0        0        0 S   0.0   0.0   0:00.00 kworker/0:0H
     6 root      20   0        0        0        0 S   0.0   0.0   0:00.87 kworker/u2:0
     7 root      20   0        0        0        0 S   0.0   0.0   0:00.82 rcu_sched
     8 root      20   0        0        0        0 S   0.0   0.0   0:00.00 rcu_bh
     9 root      rt    0        0        0        0 S   0.0   0.0   0:00.00 migration/0
    10 root      rt    0        0        0        0 S   0.0   0.0   0:00.11 watchdog/0
    11 root        0 -20        0        0        0 S   0.0   0.0   0:00.00 khelper
    12 root      20   0        0        0        0 S   0.0   0.0   0:00.00 kdevtmpfs
    13 root        0 -20        0        0        0 S   0.0   0.0   0:00.00 netns
    14 root      20   0        0        0        0 S   0.0   0.0   0:00.00 khungtaskd
```





Arquitectura *multithreaded* para servidores





Threads de *kernel* y de usuario

- ❑ **Threads de usuario** – gestionados por aplicaciones en modo usuario.
 - ❑ No necesariamente proveen paralelismo.
 - ❑ Procesador no hace cambios de contexto.
- ❑ **Threads de kernel** – gestionados por sistema operativo.
 - ❑ Son los que ejecuta el procesador.
 - ❑ Permiten paralelismo.
 - ❑ No están bajo control de aplicaciones (calendarización, sincronización, etc.).





Terminología

- Los *threads* suelen ser llamados **lightweight processes** (LWPs) mientras que un proceso tradicional es llamado **heavyweight process**.
 - El término es confuso porque en diferentes épocas y contextos ha adquirido significados diferentes.
 - A veces, los threads de *kernel* son llamados *lightweight processes* (LWPs)
 - En sistemas operativos tradicionales (System V), un LWP era una capa de información entre un *thread* de *kernel* y uno o más *threads* de usuario.
 - El sistema operativo SunOS 4 les llamaba *light weight process* a los *threads* de usuario.





Modelos de *multithreading*

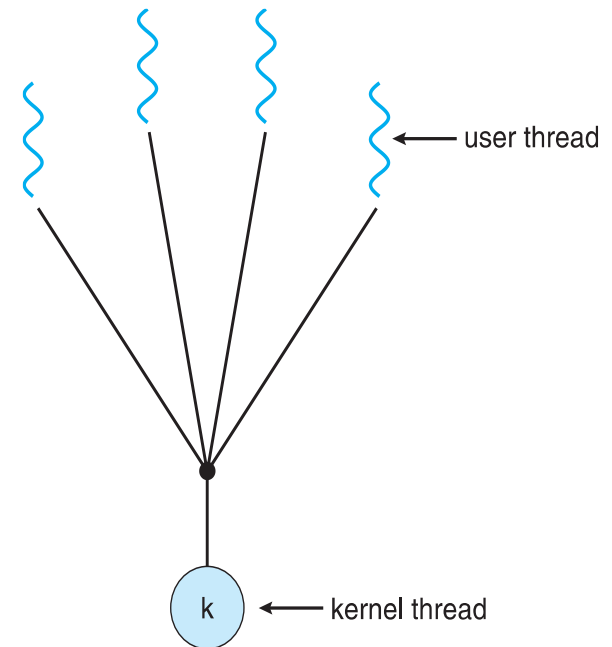
- Los modelos de *multithreading* son los siguientes:
 - Uno a uno.
 - Muchos a uno.
 - Muchos a muchos.
- Estas categorías tienen significados diferentes entre autores:
 - Silberschatz, *et al.*, se refieren a las cantidades de *threads* de usuario y de *kernel* asociadas entre sí.
 - Stallings se refiere a las cantidades de *threads* de *kernel* y “ambientes” de proceso relacionadas entre sí.
 - El *multithreading* que hemos discutido hasta ahora se basa en el caso de uno o más *threads* por ambiente de proceso.
 - ▶ Detallaremos, a continuación, las relaciones entre *threads* de usuarios y de *kernel* para este caso particular.





Muchos a uno

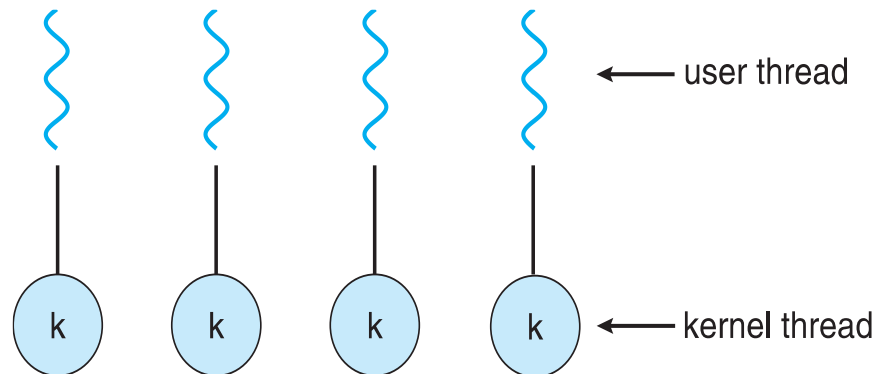
- ❑ Muchos *threads* de usuario siendo ejecutados como un único *thread* de *kernel*.
- ❑ Una llamada bloqueadora por un *thread* de usuario bloquea a los demás *threads* de usuario también.
- ❑ No permite paralelismo por medio de *threads*.
- ❑ Permite calendarización por aplicación.
- ❑ Independiente de OS.
- ❑ Ejemplos:
 - ❑ **Solaris Green Threads**
 - ❑ **GNU Portable Threads**
 - ❑ **DCE Threads**





Uno a uno

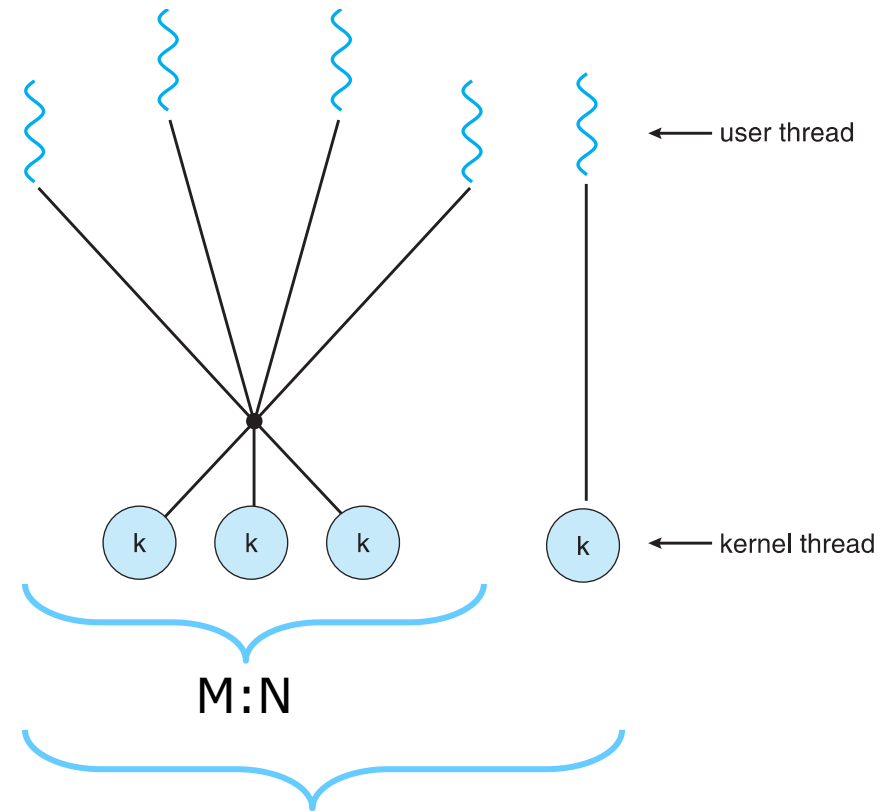
- Un *thread* de *kernel* por cada *thread* de usuario.
 - No necesariamente un proceso nuevo por cada *thread* de usuario.
- Más concurrencia (ya no bloqueos por *thread* de usuario).
- Cantidad de *threads* de *kernel* normalmente limitada, entonces los de usuario también.
- Empleado por sistemas operativos modernos como Windows con bibliotecas como WinRT, MFC y CRT; y Linux con Pthreads (NPTL).





Muchos a muchos (o *two-level*)

- Uno o más *threads* de usuario asociados con igual o menor cantidad de *threads* de *kernel*.
 - *Two-level* es el apelativo para un sistema que ofrece muchos a muchos y, además, permite asociar un *thread* de usuario a uno de *kernel* exclusivamente.
- En teoría, no presenta las desventajas de ninguno de los dos modelos anteriores.
- En la práctica pierde popularidad por el *overhead* que conlleva la creación y gestión de *threads* en ambos modos.
- Ofrecido por Windows con UMS y por HP-UX.





Scheduler Activations

- Mecanismo de comunicación entre *kernel* y *thread library* para funcionalidad muchos a muchos.
 - El *kernel* “activa” rutinas de la *thread library* ante eventos.
 - La activación efectúa una **upcall** asociada con una rutina de respuesta llamada **upcall handler**.
- Un **lightweight process** media entre uno o más *threads* de usuario y uno de *kernel*.
 - Son tratados por la *thread library* como procesadores virtuales.
 - Usan *scheduler activations* para emitir *upcalls*.

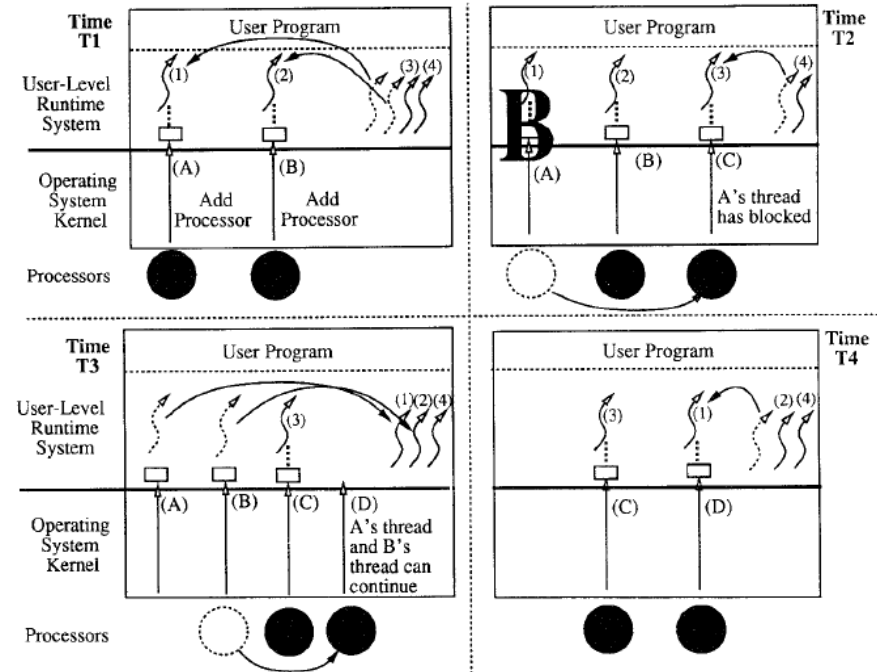


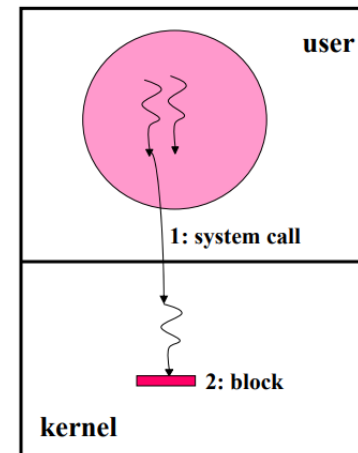
Fig. 1. Example: I/O request/completion.



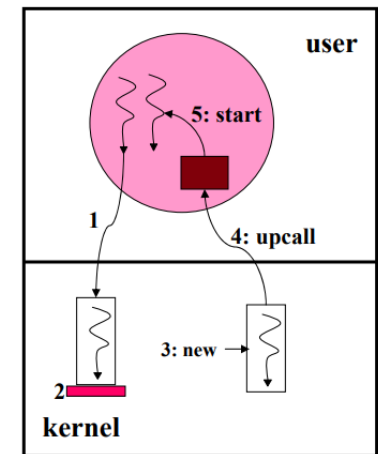


Scheduler Activations

- La *thread library* usa las *upcalls* para recalendarizar *threads* de usuario.
 1. Se bloquea el *thread* de *kernel*.
 2. Se provee otro *thread* de *kernel* con su LWP.
 3. Se emite una *upcall*.
 4. El *upcall handler* se ejecuta sobre el nuevo LWP, normalmente reemplazando el *thread* de usuario bloqueador por uno que tenga en su *ready queue*.
- *Upcalls* similares ocurren para notificar desbloques.



Kernel threads



Scheduler Activations





Los modelos de *multithreading* bajo la perspectiva de Stallings

Threads: Processes	Description	Example Systems
1:1	Each thread of execution is a unique process with its own address space and resources.	Traditional UNIX implementations
M:1	A process defines an address space and dynamic resource ownership. Multiple threads may be created and executed within that process.	Windows NT, Solaris, Linux, OS/2, OS/390, MACH
1:M	A thread may migrate from one process environment to another. This allows a thread to be easily moved among distinct systems.	Ra (Silent) , Emerald
M:N	Combines attributes of M:1 and 1:M cases.	TRIX, Clouds





Multithreading en uso

- ¿Aplicaciones de uso diario que se apoyen en *multithreading*? ¿Cómo?
- ¿Qué entenderíamos por un sistema con modelo de *multithreading* uno a muchos (1:M)?
- ¿Qué retos o dificultades surgen con la posibilidad de *multithreading* en varios núcleos (a nivel de *hardware* y de desarrollo)?





Variaciones entre *kthreads* y “procesos”

- TRIX fue un experimento en el que un mismo *kthread* podía estar relacionado con diferentes ambiente de proceso (denominados **dominios**).
 - Permitía que un hilo de ejecución “brincara” entre espacios de direcciones.
 - La separación de espacios para diferentes tareas con un mismo hilo de ejecución facilita el manejo de memoria.
 - ▶ Pero no incurre en el *overhead* de la creación de dos procesos.
- Clouds es (o era...) un sistema operativo distribuido en el que se virtualiza el espacio de memoria de un proceso.
 - Un *thread* puede brincar entre computadoras durante la ejecución.





Multithreading con varios núcleos

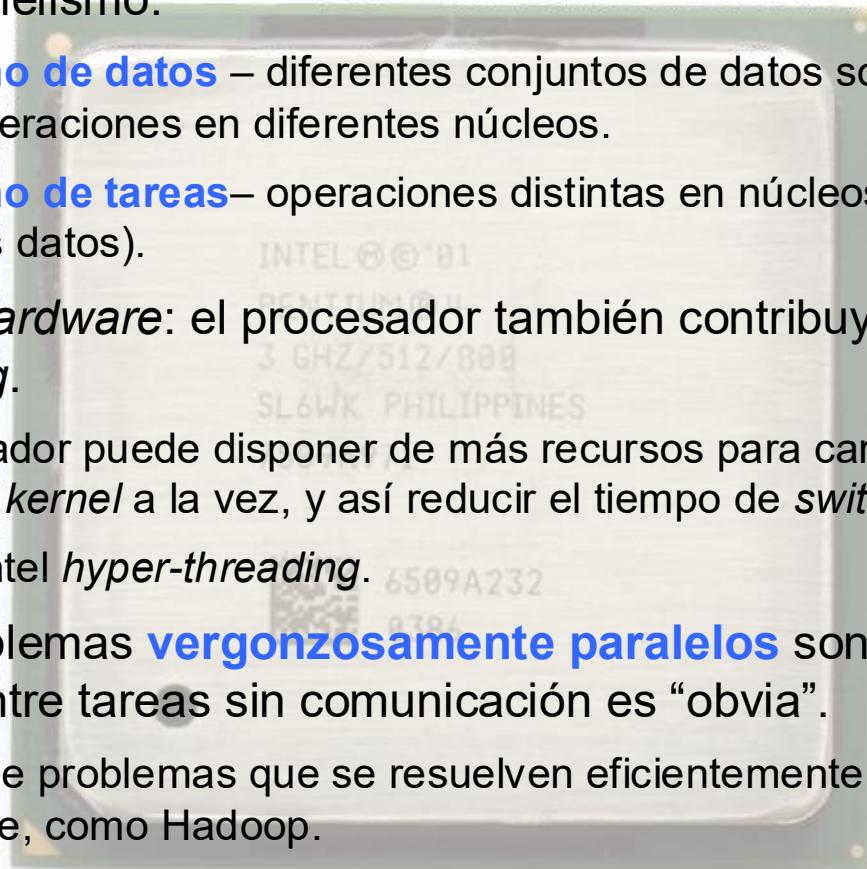
- Sistemas multinúcleo o multiprocesador presentan los siguientes retos a desarrolladores:
 - **Repartir actividades:** identificar tareas que se puedan ejecutar por separado, de forma concurrente.
 - **Balance de carga:** procesadores hacen trabajo equivalente de valor equivalente.
 - **Separación de datos:** datos accedidos por tareas concurrentes deben estar correctamente distribuidos.
 - **Dependencias:** sincronizar las tareas para acomodar su ejecución a la dependencia de información entre ellas.
 - **Testing y debugging:** con ejecución paralela hay más posibilidades que *testear*, y es más difícil seguir los pasos de ejecución.





Multithreading con varios núcleos

- Tipos de paralelismo:
 - **Paralelismo de datos** – diferentes conjuntos de datos son sometidos a las mismas operaciones en diferentes núcleos.
 - **Paralelismo de tareas**– operaciones distintas en núcleos distintos (sobre los mismos datos).
- *Threads de hardware*: el procesador también contribuye al *multithreading*.
 - Un procesador puede disponer de más recursos para cargar dos o más *threads de kernel* a la vez, y así reducir el tiempo de *switch*eo.
 - Ejemplo: Intel *hyper-threading*.
- Tareas o problemas **vergonzosamente paralelos** son problemas cuya separación entre tareas sin comunicación es “obvia”.
 - Es el tipo de problemas que se resuelven eficientemente con sistemas MapReduce, como Hadoop.





Ley de Amdahl

- Una fracción f del código o el tiempo de ejecución de un programa es paralelizable y el resto $(1 - f)$ no.
- El resultado de la fórmula indica la aceleración teórica de ejecución del programa, relativa a la ejecución sobre un único procesador.
 - Es teórica porque $\frac{f}{N}$ es la aceleración (*speedup*) ideal que tenemos al emplear N procesadores en lugar de uno sobre la parte paralelizada.
 - En realidad, mientras más procesadores, menos le sacamos el jugo a cada uno y, eventualmente, puede resultar en más *overhead* que beneficio.

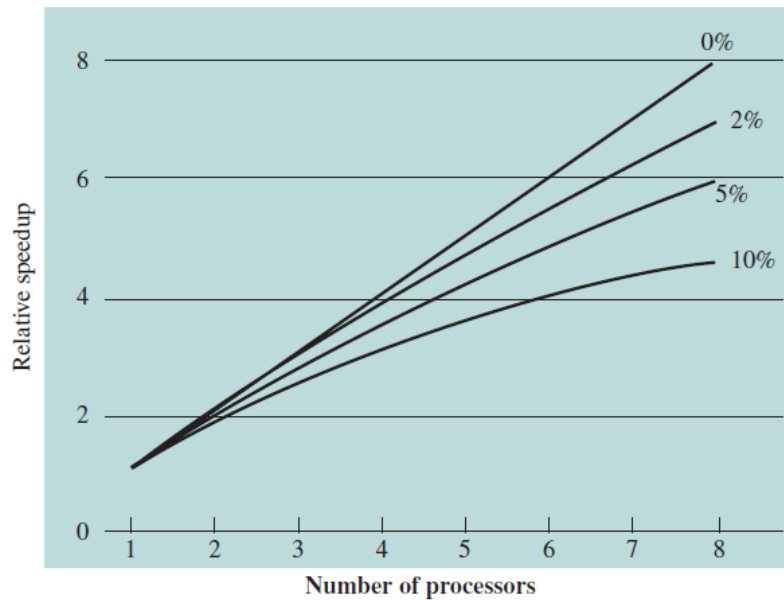
$$\text{Speedup} = \frac{\text{time to execute program on a single processor}}{\text{time to execute program on } N \text{ parallel processors}} = \frac{1}{(1 - f) + \frac{f}{N}}$$





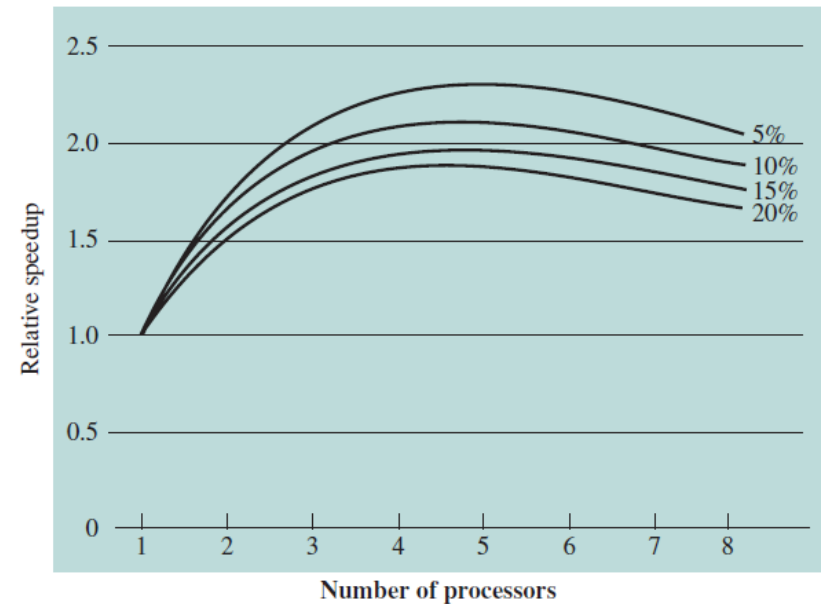
Ley de Amdahl

Speedup de programas 0, 2, 5 y 10% seriales, sin overhead



(a) Speedup with 0%, 2%, 5%, and 10% sequential portions

Speedup de programas 5, 10, 15 y 20% seriales, con overhead



(b) Speedup with overheads





Thread libraries

- Las **thread libraries** proveen API para uso de *threads*.
- Pueden ser implementadas por completo en *user space* o apoyarse en *threads* de *kernel* también.
- Ejemplos:
 - Windows: *kernel-level library*.
 - Pthreads: por ser parte del estándar POSIX, su implementación puede ser en *user* o *kernel space*.
 - Java *threads*: la JVM depende de la *thread library* del sistema operativo en que se ejecuta.
- Dos estrategias combinables para la creación de *threads*:
 - **Asincrónica**: donde el creador de *threads* continua su ejecución y cada *thread* se ejecuta de forma independiente.
 - **Sincrónica**: donde el creador manda a un montón de *threads* a hacer cosas y se queda esperando mientras los *threads* terminan.





Tecnologías relacionadas

- ¿Qué son y cómo funcionan las *fibers* en Windows?
- OpenMP es una biblioteca de *multithreading* implícito. ¿Qué provee o cómo funciona OpenMP?
- ¿Qué es y cómo funciona *Grand Central Dispatch*, la tecnología de *multithreading* desarrollada por Apple?





Threading implícito

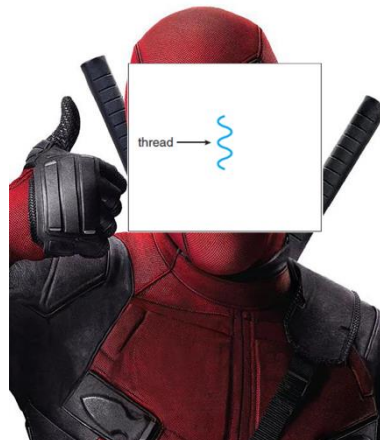
- *Threads* gestionados por compiladores y bibliotecas.
 - El programador dispone de los *threads* como un recurso, pero no necesita especificar el manejo de ese recurso.
 - Similar al beneficio provisto por las RPC's.
- Ejemplos:
 - *Thread pools*
 - OpenMP
 - *Grand Central Dispatch*





Thread pool

- Se crean los *threads* anticipadamente y se almacenan en un “banco” (*pool*), listos para ser usados.
 - Aunque consumiendo recursos “por gusto”.
- Ventajas:
 - Evita el *overhead* de creación y eliminación de *threads*. Los *threads* que terminan su trabajo son “devueltos” al banco.
 - Ofrece control sobre el límite de *threads* que la aplicación maneja.
 - ▶ Si el programa necesita un *thread* y todos están ocupados, le toca esperar.
 - Para desarrolladores se separa la parte administrativa de la parte efectiva en la programación de tareas concurrentes.





OpenMP

- Permite programación paralela en ambientes de memoria compartida (e.g., una computadora con varios procesadores o núcleos).
- Determina bloques de ejecución en paralelo.
- Si no se le limita, crea tantos *threads* como núcleos haya disponibles.
- Para usar, compilar con bandera `-fopenmp`.

```
#pragma omp parallel for
for(i = 0; i < N; i++) {
    c[i] = a[i] + b[i];
}
```

```
#include <omp.h>
#include <stdio.h>

int main(int argc, char *argv[])
{
    /* sequential code */

    #pragma omp parallel
    {
        printf("I am a parallel region.");
    }

    /* sequential code */

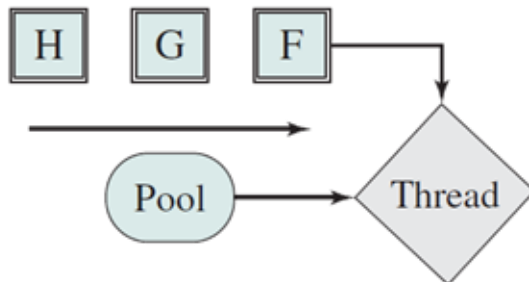
    return 0;
}
```





Grand Central Dispatch

- ❑ Tecnología desarrollada para dispositivos y computadoras de Apple.
- ❑ Permite identificar bloques de código para poner en colas.
 - ❑ Hay una cola secuencial (*main queue*) por proceso y varias colas concurrentes por sistema, separadas por prioridad.
 - ❑ Se colocan bloques en la cola secuencial para asegurar ejecución de ciertas instrucciones en secuencia.
 - ❑ Se colocan bloques en las colas concurrentes para permitir ejecución concurrente de sus contenidos. GCD decide cuántos bloques ejecuta en paralelo y en qué núcleos o procesadores.
 - ❑ Se pueden crear más colas secuenciales para un mismo proceso.
- ❑ Los *threads* con los que se descargan las colas provienen de una *thread pool*.



```
x = ^{ printf("hello world\n"); }
```





Manejo de *threads*

- ¿Qué sucede en los *threads* cuando se ejecutan llamadas que afectan al proceso entero, como `fork()` y `exec()`?
- Comentamos la llamada `kill` de sistemas *NIX ¿qué son señales para estos sistemas, y cómo se relacionan con el uso de *threads*?
 - ¿Hay señales en Windows?
- ¿Cómo se parecen los esquemas de terminación de procesos con los de cancelación de *threads*? ¿Qué son y cómo funcionan los *cleanup handlers*?





Precauciones con los *threads*

- El comportamiento de `fork()` y `exec()` en *multithreading*:
 - `exec()` reemplaza el proceso con el programa especificado en sus parámetros, *ergo* se van también sus *threads*.
 - El comportamiento de `fork()` ejecutado desde un *thread* puede variar entre sistemas operativos.
 - ▶ En sistemas apoyados en POSIX, `fork()` crea un proceso con una copia del *thread* que lo creó, nada más.





Precauciones con los *threads*

- **Signals (señales)**, en sistemas UNIX y derivados, avisan a un proceso de la ocurrencia de algún evento.
 - Sincrónicas: el proceso que ocasionó la señal también la recibe.
 - Asincrónicas: el proceso a quien se entrega la señal es distinto del que la provoca.
- Un **manejador (*handler*) de señales** reacciona ante una señal específica.
 - Los hay por defecto y definidos por usuario.
- En *multithreading*, señales sincrónicas llegan al *thread* que las genere.
- En *multithreading*, señales asincrónicas pueden llegar a todos o solo a algunos de los *threads*.
 - *Threads* pueden bloquear señales.
 - Pthreads contempla envío de señales a *threads* específicos.





Precauciones con los *threads*

- Llamada a sistema para envío de señales:
 - `kill(pid_t pid, int signal)`
- Llamada de Pthreads para envío de señales a *thread* específico:
 - `pthread_kill(pthread_t tid, int signal)`
- En Windows no hay manejo de señales, pero un *thread* puede responder ante un evento mediante APCs.
 - APC: *asynchronous procedure call*.
 - Las APC's se pueden encolar a un *thread* y el *thread* las ejecutará cuando esté en un estado "alertable".
 - ▶ Los estados alertables se consiguen mediante llamadas a funciones de espera en el *thread*.





Cancelación de *threads*

- Se refiere a detener un *thread* objetivo antes de que haga su llamada de finalización.
 - **Cancelación asincrónica**: el *thread* objetivo muere inmediatamente.
 - **Cancelación diferida** (por defecto): el *thread* objetivo revisa periódicamente si fue cancelado, y procede a terminar cuando encuentra una solicitud de cancelación en cola.
- Pthreads permite a cada *thread* determinar dinámicamente si acepta cancelaciones y si la cancelación será diferida o asincrónica.
 - La llamada para configurar “cancelabilidad” de un *thread* es `pthread_setcancelstate(int state, int *oldstate)`.
 - Solicitudes de cancelación se encolan mientras la cancelación esté deshabilitada.

```
while (1) {  
    /* do some work for awhile */  
    /* . . . */  
  
    /* check if there is a cancellation request */  
    pthread_testcancel();  
}
```



Cancelación de *threads*

- `pthread_cancel(tid)` cancela el *thread* con ID `tid`.
- `pthread_testcancel()` establece un punto para cancelación diferida.
- Cada *thread* tiene una pila de rutinas de cancelación (***cleanup handlers***).
 - `pthread_cleanup_push(void (*routine)(void *), void *arg)`
 - `pthread_cleanup_pop(int execute)`
 - Ante cancelación, se ejecutan todas, `pop` por `pop`. También cuando el *thread* termina con `pthread_exit()`.
 - Pueden ser *poppeadas* por el *thread*, decidiendo en el momento si se ejecutan o no.





Thread-Local Storage

- **Thread-local storage (TLS)** permite almacenamiento de datos locales a nivel de *thread*.
- Varía acercamiento por biblioteca de *threading* usada.
 - Pthreads usa el especificador de clase de almacenamiento `__thread` antes del tipo y nombre del elemento.
 - ▶ E.g.,

```
__thread int i;  
extern __thread struct state s;  
static __thread char *p;
```





EJEMPLOS





Threads de Linux

- ❑ Linux dice **tasks** en vez de *threads* y procesos.
- ❑ Las *tasks* se representan con estructuras de datos que apuntan a otras estructuras donde está su información.
 - ❑ Las estructuras de datos se llaman `task_struct`.
- ❑ `fork()` crea una *task*, copia su *data* y apunta su `task_struct` a las copias.
- ❑ `clone()` crea una *task*, copia datos y/o apunta la `task_struct` a los datos originales según los parámetros (*flags*) que se le pasen.
 - ❑ Con `clone()` se obtiene la funcionalidad *threads*.

flag	meaning
CLONE_FS	File-system information is shared.
CLONE_VM	The same memory space is shared.
CLONE_SIGHAND	Signal handlers are shared.
CLONE_FILES	The set of open files is shared.





Pthreads

- ❑ Usado y provisto por GNU Linux, entre otros.
- ❑ Para usar, compilar con bandera `-lpthread` o `-pthread`.

```
#include <pthread.h>
#include <stdio.h>

int sum; /* this data is shared by the thread(s) */
void *runner(void *param); /* threads call this function */

int main(int argc, char *argv[])
{
    pthread_t tid; /* the thread identifier */
    pthread_attr_t attr; /* set of thread attributes */

    if (argc != 2) {
        fprintf(stderr, "usage: a.out <integer value>\n");
        return -1;
    }
    if (atoi(argv[1]) < 0) {
        fprintf(stderr, "%d must be >= 0\n", atoi(argv[1]));
        return -1;
    }
}
```





Pthreads

```
/* get the default attributes */
pthread_attr_init(&attr);
/* create the thread */
pthread_create(&tid,&attr,runner,argv[1]);
/* wait for the thread to exit */
pthread_join(tid,NULL);

printf("sum = %d\n",sum);
}

/* The thread will begin control in this function */
void *runner(void *param)
{
    int i, upper = atoi(param);
    sum = 0;

    for (i = 1; i <= upper; i++)
        sum += i;

    pthread_exit(0);
}
```





Threads en Windows

```
#include <windows.h>
#include <stdio.h>
DWORD Sum; /* data is shared by the thread(s) */

/* the thread runs in this separate function */
DWORD WINAPI Summation(LPVOID Param)
{
    DWORD Upper = *(DWORD*)Param;
    for (DWORD i = 0; i <= Upper; i++)
        Sum += i;
    return 0;
}

int main(int argc, char *argv[])
{
    DWORD ThreadId;
    HANDLE ThreadHandle;
    int Param;

    if (argc != 2) {
        fprintf(stderr, "An integer parameter is required\n");
        return -1;
    }
    Param = atoi(argv[1]);
    if (Param < 0) {
        fprintf(stderr, "An integer >= 0 is required\n");
        return -1;
    }
}
```





Threads en Windows

```
/* create the thread */
ThreadHandle = CreateThread(
    NULL, /* default security attributes */
    0, /* default stack size */
    Summation, /* thread function */
    &Param, /* parameter to thread function */
    0, /* default creation flags */
    &ThreadId); /* returns the thread identifier */

if (ThreadHandle != NULL) {
    /* now wait for the thread to finish */
    WaitForSingleObject(ThreadHandle, INFINITE);

    /* close the thread handle */
    CloseHandle(ThreadHandle);

    printf("sum = %d\n", Sum);
}
}
```





Threads de Windows

- Cada *thread* contiene:
 - Un identificador (*thread id*).
 - Un conjunto de registros que reflejan el estado del procesador.
 - Una pila de usuario y una de *kernel* para los diferentes modos en los que el *thread* se puede estar ejecutando.
 - Un área de almacenamiento privado.
- Todo lo que almacena información (registros, área de almacenamiento y pilas) conforma el **contexto** del *thread*.





Threads de Windows

- Presentan tres estructuras de datos importantes:
 - ETHREAD (*executive thread block*) – almacena puntero al proceso engendrador, dirección de rutina de iniciación y un bloque KTHREAD.
 - KTHREAD (*kernel thread block*) – almacena información de calendarización y sincronización, la pila de *kernel* y un puntero a estructura TEB.
 - TEB (*thread environment block*) – almacena el *thread id*, la pila de usuario y el área privada.
- ETHREAD y KTHREAD almacenan información de modo *kernel*, entonces existen en ese ambiente.
- TEB se maneja y almacena información en modo usuario.





Threads de Windows

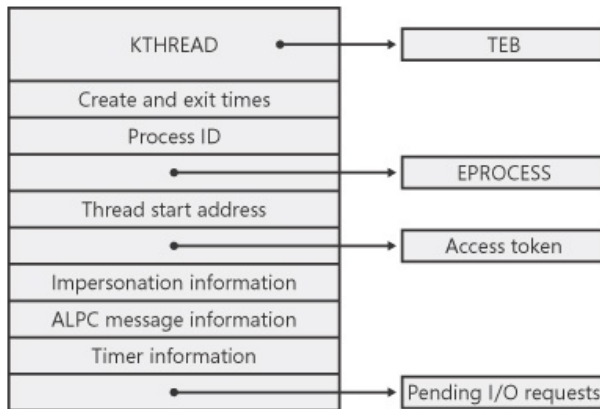


Figure 5-7. Structure of the executive thread block

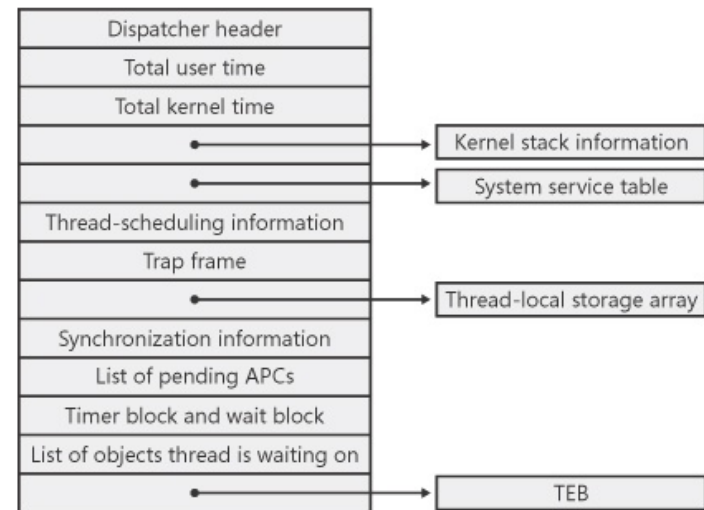


Figure 5-8. Structure of the kernel thread block

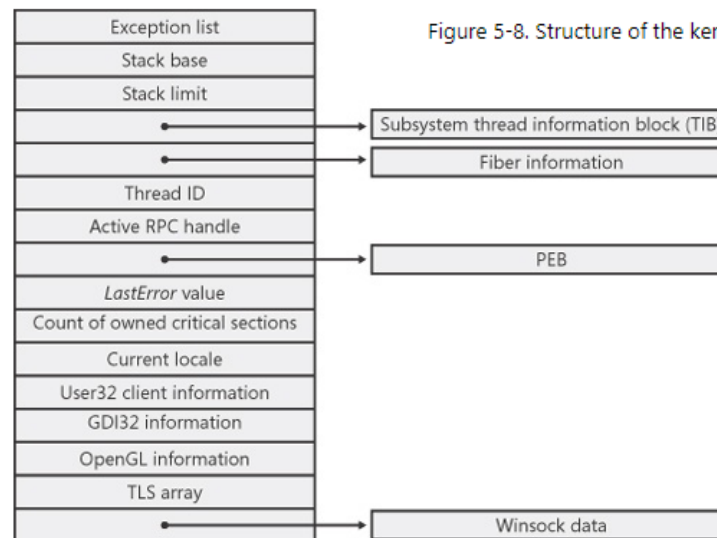


Figure 5-9. Fields of the thread environment block





Threads en Java

- Se pueden crear *threads* heredando de la clase `Thread` o implementando la interfaz `Runnable`.

```
public interface Runnable
{
    public abstract void run();
}
```

```
class Sum
{
    private int sum;

    public int getSum() {
        return sum;
    }

    public void setSum(int sum) {
        this.sum = sum;
    }
}

class Summation implements Runnable
{
    private int upper;
    private Sum sumValue;

    public Summation(int upper, Sum sumValue) {
        this.upper = upper;
        this.sumValue = sumValue;
    }

    public void run() {
        int sum = 0;
        for (int i = 0; i <= upper; i++)
            sum += i;
        sumValue.setSum(sum);
    }
}
```





Threads en Java

```
public class Driver
{
    public static void main(String[] args) {
        if (args.length > 0) {
            if (Integer.parseInt(args[0]) < 0)
                System.err.println(args[0] + " must be >= 0.");
            else {
                Sum sumObject = new Sum();
                int upper = Integer.parseInt(args[0]);
                Thread thrd = new Thread(new Summation(upper, sumObject));
                thrd.start();
                try {
                    thrd.join();
                    System.out.println
                        ("The sum of "+upper+" is "+sumObject.getSum());
                } catch (InterruptedException ie) { }
            }
        }
        else
            System.err.println("Usage: Summation <integer value>");
    }
}
```





Fuentes

- ❑ <http://www.mcs.anl.gov/~itf/dbpp/text/node10.html>
- ❑ <http://www.ni.com/white-paper/6421/en/>
- ❑ <https://www.howtogeek.com/194756/cpu-basics-multiple-cpus-cores-and-hyper-threading-explained/>
- ❑ <https://techreport.com/review/11237/valve-source-engine-goes-multi-core>
- ❑ <https://docs.oracle.com/javase/10/docs/api/java/lang/Thread.html>
- ❑ <https://docs.microsoft.com/en-us/windows/desktop/api/processthreadsapi/nf-processthreadsapi-createthread>
- ❑ https://www.tutorialspoint.com/operating_system/os_multi_threading.htm
- ❑ https://courses.cs.washington.edu/courses/cse451/11au/section/kim_a_section4.pdf
- ❑ <https://linux.die.net/man/7/pthreads>





Fuentes

- ❑ <http://courses.cs.vt.edu/cs5204/fall09-kafura/Presentations/Scheduler-Activations.pdf>
- ❑ <http://web.mit.edu/nathanw/www/userix/freenix-sa/freenix-sa.html>
- ❑ <https://computing.llnl.gov/tutorials/openMP/>
- ❑ <https://stackoverflow.com/questions/6056903/multithreaded-fork>
- ❑ <http://www.linuxprogrammingblog.com/threads-and-fork-think-twice-before-using-them>
- ❑ <https://gcc.gnu.org/onlinedocs/gcc-5.2.0/gcc/Thread-Local.html>
- ❑ <http://www.informit.com/articles/article.aspx?p=25075&seqNum=7>
- ❑ <https://www.microsoftpressstore.com/articles/article.aspx?p=2233328&seqNum=4>
- ❑ <https://docs.microsoft.com/en-us/windows-hardware/drivers/kernel/eprocess>
- ❑ <https://computing.llnl.gov/tutorials/openMP/>





Fuentes

- ❑ <https://stackoverflow.com/questions/6056903/multithreaded-fork>
- ❑ <http://www.linuxprogrammingblog.com/threads-and-fork-think-twice-before-using-them>
- ❑ <https://gcc.gnu.org/onlinedocs/gcc-5.2.0/gcc/Thread-Local.html>
- ❑ <http://www.informit.com/articles/article.aspx?p=25075&seqNum=7>
- ❑ <https://docs.oracle.com/cd/E19455-01/806-5257/mtintro-54820/index.html>
- ❑ <https://arstechnica.com/gadgets/2009/08/mac-os-x-10-6/13/>
- ❑ <http://www.ni.com/white-paper/6421/en/>
- ❑ http://www.alexonlinux.com/signal-handling-in-linux#handling_exception_signals
- ❑ <https://docs.microsoft.com/en-us/windows/win32/sync/asynchronous-procedure-calls>
- ❑ <https://h20195.www2.hp.com/v2/getdocument.aspx?docname=c04111649>





Fuentes

- ❑ <https://docs.microsoft.com/en-us/cpp/parallel/multithreading-with-c-and-win32?view=msvc-160>
- ❑ <https://docs.microsoft.com/en-us/windows/win32/procthread/fibers>
- ❑ <https://docs.microsoft.com/en-us/windows/win32/procthread/about-processes-and-threads>
- ❑ <http://www.it.uu.se/education/course/homepage/os/vt18/module-4/implementing-threads/>
- ❑ <https://lastsector.wordpress.com/2013/06/08/curious-case-of-user-mode-scheduling/>
- ❑ <https://developer.apple.com/documentation/DISPATCH>
- ❑ <https://www.zdnet.com/article/understanding-task-and-data-parallelism-3039289129/>
- ❑ https://www.cs.uic.edu/~jbell/CourseNotes/OperatingSystems/4_Threads.html#:~:text=The%20many%2Dto%2Dmany%20model,not%20block%20the%20entire%20process.





Fuentes

- ❑ Stallings, W. (2012). *Operating Systems: Internals and Design Principles*. New Jersey: Prentice Hall.
- ❑ Silberschatz, A., Galvin, P., & Gagne, G. (2013). *Operating System Concepts*. John Wiley & Sons, Inc.
- ❑ Dasgupta, P., LeBlanc Jr., R., Ahamad, M., & Ramachandran, U. (1991). The Clouds Distributed Operating System. *Computer*, 34-44.
- ❑ Goddeau, D. (1983). *A Multiple Processor Implementation of the TRIX Operating System*. Massachusetts: MIT.
- ❑ Scheduler Activations: Effective Kernel Support for the User-level Management of Parallelism. (1992). *ACM Transactions on Computer Systems*, Vol. 10, No. 1, 53-79.
- ❑ Vahalia, U. (1996). *UNIX Internals: the new frontiers*. New Jersey: Prentice Hall.
- ❑ Mohsin, Ahmad & Naqvi, Irfan & Fatima, Syeda. (2014). Thread models Semantics: Solaris and Linux M:N to 1:1 thread model. 10.13140/RG.2.1.4608.9767.

